

12 tuần chuyển hóa internship thành năng lực tự làm

Career Playbook v4.0

Ba lane: công ty, full SWP391 và technical evidence

Zero-ready teaching • runnable labs • work evidence • mastery gates • canonical Laundry transfer

Personalized self-study edition v4.0 • August 2026

Mục lục

Career Playbook v4	4
1. Playbook này giải quyết bài toán gì?	5
2. Định vị nghề nghiệp sau 12 tuần	7
3. Market evidence: nhà tuyển dụng đang hỏi gì?	8
4. Skill priority: học sâu cái gì, học đủ cái gì?	10
5. Cách vận hành tuần học	12
6. Roadmap 12 tuần	14
Tuần 1 - QA thinking, môi trường và JavaScript đầu tiên	15
Tuần 2 - Function, data, module và web anatomy	17
Tuần 3 - Async, test oracle và cầu nối sang Playwright	18
Tuần 4 - SWP kickoff: canonical map, auth và catalogue/customer slices	20
Tuần 5 - API/HTTP skill + full access/catalogue/customer implementation	22
Tuần 6 - Order vertical core: UC-09-14	23
Tuần 7 - Processing, lifecycle và issues: UC-15-19	24
Tuần 8 - SQL, immutable ledger và pickup: UC-20-24	25
Tuần 9 - Public tracking, dashboard, reports, settings và pickup schedule: UC-25-30	26
Tuần 10 - Cross-cutting CI/CD, persistence và canonical integrity	27
Tuần 11 - Full canonical delivery, defense và automation capstone	28
Tuần 12 - Consolidation: independent transfer + full SWP defense	29
7. CI/CD hay Docker trước?	30
8. Portfolio architecture	31
9. Skill evidence log - CV use là tùy chọn	33
10. Interview drills	34

11. English lane	36
12. Branching rules theo deadline và thực tế internship	37
13. Sau 12 tuần nên học sâu automation hay rẽ sang CI/CD/Docker?	39
14. Calibration questionnaire trước khi khóa v4	40
15. Definition of ready sau 12 tuần	42
16. Quy tắc giữ v4 trung thực và hiệu quả	43
17. One-page priority card	44

Career Playbook v4

Lộ trình 12 tuần: SWP391 Laundry + Playwright thực chiến + đường vào việc QA có lương

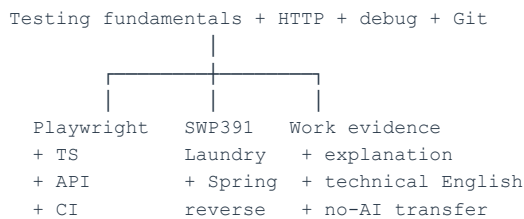
*Trạng thái: bản phát hành Self-Study v4 Ngày hiệu chỉnh thị trường: 13/08/2026 Đối tượng đã hiệu chỉnh: người tự xem mình là **true zero** sau bốn năm phụ thuộc AI, còn gần ba tháng internship; SWP391 bắt đầu sau khoảng ba tuần và kéo dài hai tháng*

1. Playbook này giải quyết bài toán gì?

Người học không có một mục tiêu duy nhất. Trong cùng 12 tuần, họ cần:

1. **Tự làm full SWP391:** hiểu, xây và bảo vệ được toàn bộ Laundry đúng canonical scope, không chỉ phần được chia hay code do AI sinh.
2. **Làm được việc Playwright tại công ty:** đọc task, phân tích rủi ro, viết test ổn định, debug failure và giao tiếp bằng evidence.
3. **Có năng lực đủ thật để tự tìm việc có lương:** sở hữu profile kỹ năng Junior QA/QC đủ rộng, có bằng chứng automation thật và không phụ thuộc vào việc nhà tuyển dụng có đúng title "Playwright Automation Tester". Việc tìm JD, nộp hồ sơ và quản lý application do người học tự thực hiện; curriculum tập trung huấn luyện năng lực.

Ba mục tiêu này có phần giao nhau, nhưng không hoàn toàn trùng nhau:



Vì vậy v4 không tổ chức như ba khóa học riêng biệt. Nó dùng nhịp **3 + 8 + 1**:

- **Ba tuần runway mở đầu:** phá phụ thuộc AI và đi foundation theo đúng thứ tự; đây là ba tuần lịch trước SWP, không phải lời hứa mọi gate JS/TS/Playwright đều xong trong 30 giờ.
- **Tám tuần full-SWP integration:** học toàn bộ canonical Laundry theo vertical slices, đồng thời dùng chính các flow đó để luyện Playwright, API/HTTP, SQL, Git và CI.
- **Một tuần consolidation:** random change, seeded bug, full canonical defense và career-skill assessment.

Reality check về thời lượng

Tổng timebox thường gặp của QA00 + JS01-JS08 + TS01-TS04 là khoảng **55-62 giờ**, chưa tính Playwright P01-P11. Vì vậy:

- nhịp Standard 8-10 giờ/tuần cần khoảng 6-7 tuần chỉ để qua toàn bộ Part I;
- nhịp Survival 4 giờ/tuần sẽ lâu hơn nhiều;
- SWP bắt đầu sau ba tuần lịch thì foundation **tiếp tục song song**, không được nhảy prerequisite;
- task công ty có thể được đọc/trace/review sớm, nhưng patch có mentor/AI trước gate được ghi

assisted, không tính là tự làm được.

Nếu muốn hoàn tất Part I đúng ba tuần, arithmetic đòi gần 18-21 giờ/tuần. Playbook không mặc định m có bandwidth đó bên cạnh internship; chất lượng gate được giữ, deadline học được co giãn.

Mỗi tuần có một **trunk skill** dùng chung, rồi chuyển cùng skill đó sang:

- một task hoặc tình huống giống công ty;
- một vertical slice của Laundry;
- một artifact có thể dùng làm bằng chứng nghề nghiệp.

Cam kết thực tế

Playbook này không hứa chắc có việc sau 12 tuần. Thị trường Cần Thơ/TP.HCM, chất lượng internship, cơ hội mở đúng thời điểm và khả năng phỏng vấn đều ảnh hưởng đến kết quả.

Mục tiêu hợp lý sau 12 tuần là:

Có thể ứng tuyển trung thực vào nhóm Fresher/Junior QA/QC, Manual QA có automation, Junior Tester và một phần Junior Automation Tester; đồng thời có đủ evidence để nhà tuyển dụng phân biệt mình với ứng viên chỉ học tutorial.

“Fresher” hay “Junior” chỉ là nhãn tuyển dụng, không phải learning gate. Gate của v4 là việc người học tự giải thích, sửa biến thể và debug được mà không có AI làm hộ. Không dùng ba tháng internship để tự nhận là có “ba năm kinh nghiệm”. Không bịa impact, không đưa code hoặc dữ liệu mật của công ty vào evidence. IELTS khoảng 6.5 dù chưa có chứng chỉ đã đủ để không cần mở một khóa English tổng quát; English lane chỉ luyện technical reading, explanation và interview transfer.

2. Định vị nghề nghiệp sau 12 tuần

Định vị chính

Junior QA/QC có năng lực Playwright + TypeScript, hiểu API/SQL/Git/CI và có trải nghiệm dự án thực tế.

Đây là định vị rộng vừa đủ để không tự khóa mình vào một title hiếm, nhưng vẫn có một điểm mạnh kỹ thuật rõ ràng.

Nhóm job nên nhắm

Ưu tiên theo thứ tự:

1. Fresher/Junior QA Engineer hoặc QA/QC có cả manual và automation.
2. Junior Tester yêu cầu Web/API/SQL/Git và coi automation là lợi thế.
3. Junior Automation Tester dùng Playwright, Cypress hoặc framework tương tự.
4. Manual QA trong một team có đường chuyển sang automation rõ ràng.

Chưa nên tự định vị là:

- SDET độc lập thiết kế framework cấp công ty;
- DevOps/CI engineer;
- Java/Spring backend developer;
- performance, security hoặc mobile automation specialist.

Câu giới thiệu 30 giây mục tiêu

Em là Junior QA/QC tập trung vào web testing. Trong internship và dự án SWP391, em luyện cách chuyển requirement thành test conditions, viết Playwright bằng TypeScript, kiểm tra API và dữ liệu, debug bằng trace/log, rồi chạy regression trên CI. Em có thể trình bày rõ vì sao chọn test nào, failure nằm ở product hay test, và evidence nào chứng minh kết luận đó.

Phải điều chỉnh câu này theo bằng chứng thật. Không nói đã dùng API, SQL hoặc CI trong internship nếu chỉ dùng chúng trong portfolio.

3. Market evidence: nhà tuyển dụng đang hỏi gì?

Phần này chỉ dùng để quyết định **học gì**, không biến v4 thành khóa sẵn việc. Snapshot được kiểm tra ngày 13/08/2026; tin có thể đồng hoặc đổi sau ngày đó, nên đây là mẫu năng lực chứ không phải danh sách chỗ để nộp.

Evidence 1 - Entry level bắt đầu từ testing và một ngôn ngữ

SmartOSC tuyển Automation Tester Fresher nhưng phần việc vẫn bắt đầu từ phân tích requirement, viết test case/test plan, regression và bug report; yêu cầu là nền tảng testing/SDLC, quen ít nhất một ngôn ngữ và English. CI/CD được đào tạo trong quá trình làm việc.

Nguồn: [SmartOSC - Automation Tester Fresher](#)

Suy ra cho v4: không được dạy Playwright như công cụ record-click. Requirement, test design, regression, bug reporting và programming foundation đi trước framework.

Evidence 2 - SQL, API và English có giá trị ngay ở fresher

Tin Fresher Automation Test của FPT yêu cầu kiến thức cơ bản về một ngôn ngữ, manual testing/testing techniques và SQL; automation framework, API/Postman/JMeter là ưu tiên. English tương đương TOEIC 600 hoặc IELTS 6.0 cũng được nêu như một lợi thế.

Nguồn: [FPT Software - Fresher Automation Test](#)

Suy ra cho v4: API, SQL và English là lane hỗ trợ có giá trị cao; chúng không nên bị đẩy ra sau một khóa Playwright thuần UI.

Evidence 3 - Fresher/Junior vẫn là QA trước khi là framework author

OPSWAT mô tả role Fresher/Junior thực thi manual cases, làm rõ scenario từ requirement, viết test documentation, test functional/integration/API/regression/usability và ghi bug rõ. Git/CI/CD và automation exposure nằm ở nhóm nice-to-have.

Nguồn: [OPSWAT - QA Automation Engineer Fresher/Junior](#)

Suy ra cho v4: CI quan trọng nhưng chưa được phép chiếm thời gian của test fundamentals, HTTP/API và khả năng giải thích failure.

Evidence 4 - Đích Playwright sau entry cần Git, CI và debug

Một role Playwright của RightShip (yêu cầu tối thiểu hai năm, nên chỉ dùng làm đích năng lực) kết hợp Playwright TypeScript/JavaScript với DOM/browser, test methodology, manual regression/UAT, Git/GitHub, CI pipeline, troubleshooting và English.

Nguồn: [RightShip - QA Automation Engineer](#)

Suy ra cho v4: cuối lộ trình phải có một pipeline thật và debug evidence; chưa cần tự nhận là SDET hay framework engineer.

Docker chưa lập lại trong các mẫu entry trên. Nó xuất hiện như năng lực môi trường ở một role HCM yêu cầu hơn bốn năm kinh nghiệm, cùng API/UI/CI/CD. Vì vậy v4 chỉ dạy Docker ở mức vận hành, không rõ DevOps.

Nguồn đối chiếu mức sau: [WAO HCM - QA Automation Engineer](#)

Kết luận từ market evidence

Trật tự tạo employability hợp lý là:

```
Testing thinking
→ JavaScript/TypeScript
→ Playwright ổn định và debug
→ HTTP/API
→ SQL
→ Git
→ CI/CD
→ Docker cơ bản
```

Nguồn kỹ thuật dùng để kiểm tra behavior trong khóa học nên ưu tiên tài liệu chính thức:

- [Playwright Best Practices](#)
- [Playwright CI](#)
- [Playwright Locators](#)
- [TypeScript Handbook](#)
- [Git Reference](#)

4. Skill priority: học sâu cái gì, học đủ cái gì?

Năng lực	Mức sau 12 tuần	Vì sao	Evidence tối thiểu
Testing fundamentals	Sâu ở mức junior	Là nền của manual, automation và interview	Test conditions, boundary/negative cases, risk matrix, bug reports
JavaScript	Vững phần dùng cho test	Nếu chỉ copy syntax Playwright thì không debug được	Tự viết function, array/object transforms, async flow, error handling
TypeScript	Vững phần dùng cho automation	Job Playwright thường đi cùng TS	Type inference, object type, union, optional, narrowing, module, đọc compiler error
Web/HTTP/DOM	Vững	Locator, API, auth, redirect và Spring đều dựa trên lớp này	Giải thích request/response, status, cookie/session, semantic DOM
Playwright core	Sâu	Điểm mạnh định vị nghề	Stable UI suite, fixture, auth, trace/debug, CI artifact
Debugging	Sâu	Tạo khác biệt lớn hơn số lượng test	Hypothesis/evidence log và ba seeded-failure repairs
API testing	Làm được độc lập ở mức junior	Xuất hiện thường xuyên trong JD	8-12 tests, auth/error/schema/data checks
SQL/PostgreSQL	Làm được truy vấn kiểm chứng	Nhiều job junior yêu cầu	10-12 queries có join/group/aggregate và business oracle
Git	Làm việc nhóm được	Internship và mọi project đều cần	Branch, commit có nghĩa, PR, resolve conflict đơn giản, đọc diff
CI/CD	Vận hành một pipeline được	Bằng chứng test chạy ngoài máy cá nhân	Pipeline xanh, artifact, deliberate failure và triage note
Docker	Biết dùng, chưa cần thiết kế hạ tầng	Hữu ích cho môi trường nhưng không phải bottleneck đầu tiên	Run/stop/log/env/volume/compose ; chạy app + DB nếu project đã dùng
Java/Spring	Đọc và trace được	Cần cho SWP391, không phải nhánh nghề chính 12 tuần	Trace Controller → Service → Repository → DB → response và test
English QA	Dùng được mỗi tuần	Tài liệu, ticket và interview thường cần	Bug report, stand-up, test explanation và README ngắn bằng English
Evidence/interview transfer	Đủ chứng minh kỹ năng	Chuyển learning thành tín hiệu có thể kiểm tra	Clean-run repo, README, demo, evidence log, mock technical interview

Chưa học sâu trong 12 tuần

- Kubernetes và cloud platform.
- Docker networking/production hardening nâng cao.
- Selenium chỉ để “có thêm framework”.
- Appium/mobile nếu internship và JD mục tiêu không yêu cầu.
- Performance/security chuyên sâu; chỉ học awareness và test ideas.
- Framework abstraction, base class hoặc Page Object quá mức.
- DSA nặng, design pattern catalog và Java backend chuyên sâu.

- Chứng chỉ chỉ để trang trí CV.

5. Cách vận hành tuần học

Một “tuần” trong playbook là một **module có gate**, không nhất thiết phải trùng thứ Hai đến Chủ nhật. Khi cuộc sống lệch nhịp, không học lại từ đầu.

Nhịp Standard - 8 đến 10 giờ

- 3-4 giờ teaching + lab cho trunk skill.
- 2 giờ company transfer.
- 2-3 giờ Laundry/SWP transfer.
- 1 giờ retrieval, English và career evidence.

Tỷ trọng đổi theo phase:

- **Tuần 1-3:** khoảng 50% zero foundation, 25% company transfer, 15% pre-SWP orientation và 10% retrieval/English.
- **Tuần 4-11:** khoảng 45% canonical SWP vertical slice, 30% shared automation skill, 15% company transfer và 10% retrieval/evidence.
- **Tuần 12:** consolidation/gates; không mở concept lớn mới.

Nhịp Survival - khoảng 4 giờ

- 90 phút core lesson/lab.
- 60 phút company transfer.
- 60 phút SWP artifact.
- 30 phút retrieval + evidence log.

Stretch task bị cắt trước; gate và evidence không bị cắt.

Nhịp Sprint - tối đa 12 giờ

Chỉ dùng khi SWP, internship hoặc interview có deadline gần. Không chạy quá hai module liên tiếp để tránh tạo một kế hoạch đẹp nhưng không bền.

Ngay cả nhịp Sprint cũng không đủ biến 55-62 giờ foundation thành ba tuần. Sprint ưu tiên current gate và cắt stretch task; nó không cho phép bỏ JS06 rồi viết `await`, hoặc bỏ TS04 rồi tự nhận đã sẵn sàng Playwright.

Learning loop bắt buộc

Mỗi concept mới dùng cùng một chuỗi:

```
Mental model ngắn
→ worked example có chú thích
→ dự đoán output/behavior
→ guided lab
→ tự viết biến thể
→ failure injection
→ company transfer
→ Laundry transfer
→ delayed retrieval
```

Không qua gate chỉ vì test đang xanh. Người học phải giải thích được, sửa được biến thể và debug được failure.

AI-dependence recovery contract

Vì baseline thực tế là “true zero do bốn năm để AI làm hộ”, v4 đánh giá kỹ năng bằng transfer chứ không bằng tốc độ tạo code.

Mỗi lab dùng bốn mode theo đúng thứ tự:

1. **Closed book, 10-20 phút:** dự đoán, viết pseudo-code hoặc đưa hypothesis mà không hỏi AI.
2. **Docs/search, 10-20 phút:** tra syntax/behavior nhưng chưa xin lời giải hoàn chỉnh.
3. **Socratic hint:** chỉ xin một hint nhỏ nhất cho blocker đã mô tả.
4. **Evidence review:** sau khi code/test chạy, nhờ AI review reasoning, edge cases hoặc diff.

Các bài gate, random change và seeded bug không cho AI sinh patch đầu tiên. Mỗi tuần phải có ít nhất:

- một đoạn code tự gõ từ file trống;
- một prediction ghi trước khi chạy;
- một failure được debug bằng hypothesis/evidence;
- một teach-back không nhìn tài liệu;
- một biến thể mới khác guided example.

Nếu chỉ có code xanh do AI tạo, evidence tuần đó được đánh dấu **assisted**, không được tính là passed gate.

6. Roadmap 12 tuần

Phase	Tuần	Trọng tâm
Zero-to-ready runway starts	1-3	Testing + ordered JS foundation; async/TS/Playwright tiếp tục theo gate nếu chưa xong
Full canonical SWP integration	4-11	Đi hết UC-01-30 và BR-01-39 theo vertical slices; cài API/SQL/Git/CI/Docker đúng thời điểm
Consolidation	12	Random change, seeded failure, full defense và next-skill decision

Tuần 1-3 không cố “học xong lập trình”. Mục tiêu là tạo runway thật và một restart line chính xác. Nếu module còn open khi SWP bắt đầu, tiếp tục đúng ID ở tuần 4-6 song song với canonical map/J00; không đổi thứ tự chỉ để bảng lịch trông đẹp.

Tuần 1 - QA thinking, môi trường và JavaScript đầu tiên

Outcome

- Biết software testing đang giảm rủi ro gì.
- Chuyển một requirement ngắn thành test conditions.
- Viết và chạy được expression/decision/loop JavaScript nhỏ bằng Node.
- Dùng Git để lưu evidence có lịch sử rõ ràng.

Teaching layer

- Expected vs actual; oracle; severity vs priority.
- Positive, negative, boundary, equivalence partition.
- Requirement ambiguity và câu hỏi làm rõ.
- Bug lifecycle và cấu trúc bug report.
- QA00: terminal, file/folder, Node, npm, `package.json`, script và failure layer.
- JS01: values, variables, conversion và expressions; chưa dùng `if`/function/object.
- JS02: condition, boolean decision và `for...of`; chưa dùng function/array method.
- JS03 chỉ bắt đầu nếu QA00-JS02 đã qua gate và còn bandwidth.
- Git working tree, stage, commit và diff.

Company transfer

Chọn một feature nhỏ đang gặp ở internship. Viết:

- 5 risk statements;
- 10-15 test conditions;
- 3 bug reports mẫu hoặc ba failure observations có expected/actual/evidence.

Nếu bị ràng buộc bảo mật, dùng mô tả đã ẩn tên product/data.

Laundry transfer

Chọn UC-05 hoặc UC-25. Viết actor, goal, precondition, happy path, negative cases và business oracle. Không thêm customer account hoặc feature ngoài canonical scope.

Deliverable

- QA00, JS01 và JS02 completion + failure evidence; JS03 là carry-over hợp lệ.
- 8-12 micro-exercises bám đúng syntax đã học, không lấy số lượng làm gate.
- Một test-design note.
- Ba bug reports.
- Repo chạy được bằng README ngắn.

Gate

Người học phải tự:

1. Giải thích khác nhau giữa test case và bug report.
2. Viết role/status decision không lấy expected từ actual; nếu đã tới JS03, thêm một function contract.

3. Dùng `git diff` để chỉ ra thay đổi trước khi commit.

Tuần 2 - Function, data, module và web anatomy

Outcome

- Xử lý array/object đủ để đọc dữ liệu test.
- Hoàn thành function/array/object/module foundation trước TypeScript.
- Đọc được semantic DOM và một HTTP exchange cơ bản.
- Đọc anatomy test có sẵn; chưa tự viết Playwright trước JS08 + TS04.

Teaching layer

- JS03: function, parameter/argument, return, scope và callback.
- JS04: array, object, reference, destructuring và spread.
- `map`, `filter`, `find`, `some`, `every`, `reduce` ở mức cần dùng.
- JS05: module, `import/export`, JSON, stack trace và error; giữ file read

đồng bộ vì Promise thuộc JS06.

- HTML semantics, form controls, accessible name.
- URL, method, status, header, body, cookie và redirect.
- Cấu trúc `test`, locator, action, assertion chỉ ở mức đọc symbol/risk, chưa tính P gate.

Company transfer

Đọc một spec trong repo công ty. Với mỗi dòng quan trọng, ghi:

- data có type gì;
- locator đang dựa vào semantic nào;
- assertion kiểm tra business outcome hay chỉ kiểm tra implementation detail;
- request nào có khả năng xảy ra sau action.

Laundry transfer

- Trace UC-05 và UC-25 từ UI behavior tới HTTP request/response.
- Chỉ ra dữ liệu public nào được phép hiển thị và dữ liệu nào không được leak theo BR-09.

Deliverable

- JS03-JS05 completion/independent/failure evidence theo gate.
- 8-12 data/module exercises và một deliberate import/JSON failure note.
- Một request/response anatomy note.

Gate

- Viết function xử lý array of objects mà không xem đáp án.
- Tách module không dùng async syntax trước JS06 và đọc first owned stack frame.
- Giải thích business identity khác index thế nào; locator chỉ là transfer analogy ở giai đoạn này.

Tuần 3 - Async, test oracle và cầu nối sang Playwright

Outcome

- Hiểu callback, Promise, `async/await` và error propagation.
- Viết Node tests có oracle độc lập và đọc anatomy Playwright qua JS08.
- Bắt đầu TypeScript strict; chỉ viết browser spec khi đã qua TS04.

Teaching layer

- Synchronous stack, task, microtask ở mức thực dụng.
- Promise states, missing `await`, sequential vs concurrent work.
- JS07: Arrange/Act/Assert, boundary, mutation và oracle.
- JS08: registration/callback/fixture, business locator plan và awaited steps.
- TS01 rồi TS02-TS04 theo gate; phần TypeScript chưa xong được carry sang tuần 4-6.
- Actionability, re-render và web-first assertion chỉ bắt đầu sau TS04/P01.
- Khi nào cần `Promise.all`; khi nào không.
- Vì sao `waitForTimeout` che lỗi thay vì sửa lỗi.

Company transfer

Đọc một test flake hoặc test có wait cứng được phép. Viết trước:

1. expected;
2. actual;
3. ba hypothesis;
4. evidence sẽ thu;
5. thay đổi nhỏ nhất để kiểm chứng.

Nếu chưa qua TS04, dừng ở diagnosis/plan và nhờ reviewer thực hiện patch công ty; không dùng task thật để tự cấp gate Playwright.

Laundry transfer

Tạo state-transition test matrix cho:

```
received → processing → ready_for_pickup → completed
      ↙
      L cancelled
```

Bao gồm invalid transitions và open-issue/payment guards.

Deliverable

- JS06-JS08 completion + Node test evidence.
- TS01 completion; TS02-TS04 có thể là carry-over theo bandwidth.
- Một Playwright test chỉ khi TS04 đã pass; nếu chưa, nộp anatomy/locator/oracle plan.
- Một missing-`await` repair note.
- Một flake investigation report.

Gate

- Vẽ dependency graph và sửa missing `await` bằng evidence.
- Cài mutation và chứng minh Node suite bắt bug.
- Giải thích registration/fixture/action/assertion; Playwright stability gate được chuyển đúng sang P01-P03.

Tuần 4 - SWP kickoff: canonical map, auth và catalogue/customer slices

Outcome

- Bắt đầu SWP với một bản đồ đầy đủ thay vì lao ngay vào CRUD.
- Hiểu scope, actor, exclusion, fixed stack và traceability contract.
- Xây/đọc được access, catalogue và customer flows đầu tiên.
- Dùng fixture/auth state và Git collaboration trên code thật.

Teaching layer

Lane foundation song song: tiếp tục đúng lesson open từ tuần 3. Phải qua TS04 trước P01. Nếu chưa qua, tuần này vẫn làm canonical map, J00 Java reading, requirement/test design và Spring trace; browser patch có hỗ trợ không được tính mastery.

- Canonical baseline: 30 UCs, 39 BRs, 5 order statuses, 2 issue statuses, 9 tables.
- Java-reading essentials: class, method, object, exception, annotation và generic type thường gặp.
- Browser → HTTP → Filter/Security → DispatcherServlet → Controller mental map.
- Test isolation, fixture scope, authentication vs authorization, session và CSRF.
- Git branch, focused commit, diff, PR và review.

Full-canonical checkpoint

- Đọc và lập one-line purpose cho UC-01-30.
- Nhóm BR-01-39 theo identity, customer, catalogue, order, issue, money, report và persistence.
- Gắn mỗi table vào business purpose; không tạo table do framework gợi ý.
- Liệt kê explicit exclusions: customer account, delivery, inventory, promotion, multi-branch, React/JWT/REST-first và các feature ngoài MVP.

Vertical slices tuần này

- UC-01-04: login, logout, change password, Staff administration.
- UC-05-06: public catalogue và Manager service maintenance.
- UC-07-08: customer contact và order history.
- Tập trung BR-01-13; các rule liên quan được dùng lại ở tuần sau.

Company transfer

Chọn một authenticated flow trong internship. Thiết kế role, setup, data ownership và failure evidence. Nếu repo công ty có fixture/auth state, đọc rồi viết lại mental model bằng lời của mình.

Deliverable

- Canonical one-page map.
- Permission matrix Guest/Staff/Manager.
- Auth/data fixture hoặc guided implementation.
- Một PR có self-review checklist.
- 6-10 UI tests cho public/auth/catalogue/customer flows nếu app đã runnable.

Gate

- Nói được toàn bộ product boundary và vì sao các exclusion tồn tại.
- Hai tests đổi thứ tự vẫn pass.
- Giải thích vì sao hidden button không thay server authorization.
- No-agent: trace một login request đến security layer mà không nhìn đáp án.

Tuần 5 - API/HTTP skill + full access/catalogue/customer implementation

Outcome

- Hoàn thành hoặc audit UC-01-08 theo vertical trace.
- Thiết kế API/HTTP checks từ contract/risk, không chỉ kiểm tra 200.
- Hiểu form POST, validation, CSRF và PRG của server-rendered Laundry.

Teaching layer

- HTTP method, status, header, cookie, body và redirect sâu hơn.
- REST/API testing trên demo hoặc company API: auth, schema/shape, boundary và side effect.
- Spring binding, Bean Validation, service guard và error presentation.
- Thymeleaf form, CSRF token, RedirectAttributes và Post/Redirect/Get.
- MockMvc vs browser E2E: mỗi lớp chứng minh điều gì.

J02 có gate hai chặng: điền HTTP matrix ở tuần này; phần executable `tests/work/p08_api.spec.ts` chỉ mở sau TS04 và P01-P07. Không nhảy thẳng tới P08 chỉ vì roadmap đang nói API.

Company transfer

Nếu công ty có API được phép test, tạo 8-12 tests cho một resource. Nếu chưa có access, dùng demo API và chuyển reasoning sang HTTP flow thực tế của công ty.

Laundry transfer

Canonical Laundry là server-rendered Spring MVC. Không thêm REST endpoint, React client hay JWT chỉ để luyện API. Với UC-01-08:

- trace form/query → Controller → service → repository/table → view/redirect;
- kiểm tra authorization, CSRF, validation và historical-retention rules;
- giữ UI tiếng Việt và identifiers kỹ thuật tiếng Anh.

Deliverable

- UC-01-08 traceability bundle.
- 8-12 API tests trên target phù hợp.
- MockMvc/browser checks cho happy, invalid-input và forbidden cases.
- Một failed-response classification note.

Gate

- Giải thích được 400/401/403/404/409 theo contract cụ thể.
- Giải thích vì sao 200 không nghĩa business outcome đúng.
- Random role change: dự đoán đúng affected route/service/test trước khi sửa.

Tuần 6 - Order vertical core: UC-09-14

Outcome

- Hiểu và làm được create/view/search/revise/cancel/receipt order flow.
- Theo request từ browser xuống PostgreSQL rồi quay lại view/redirect.
- Viết Playwright tests có isolation, data strategy và business oracle rõ.

Teaching layer

- DispatcherServlet, typed form/query binding, validation và Controller responsibility.
- Focused service method, `@Transactional`, repository/JPA, entity và Flyway.
- Snapshot semantics, BigDecimal, rounding và order-code generation.
- Form complete-line semantics và Post/Redirect/Get.
- Fixture/data builder khi domain data phức tạp; tránh Page Object quá mức.

Canonical Laundry transfer

- UC-09-14.
- BR-14-20, đồng thời áp lại BR-07-13.
- UC-09 create order và payment là hai transaction/actor actions tách biệt.
- Order revision chỉ khi `received`; giữ snapshot; new service phải active; omitted line bị remove.
- Cancellation không hard-delete và có refund-due semantics.

Company transfer

Chọn một CRUD/workflow flow thật. Tạo matrix happy, invalid input, stale/tampered data và authorization. Nếu target có API setup, dùng nó để seed data nhưng UI vẫn chứng minh user outcome.

Deliverable

- Full reverse trace cho UC-09 và một UC trong 10-14.
- 8-12 Playwright tests cho order core.
- Snapshot/rounding test table.
- Một seeded tampered-ID hoặc inactive-service failure investigation.

Gate

- Trace UC-09 không nhìn tài liệu trong 8 phút.
- Giải thích vì sao Controller không tính giá/balance.
- No-agent random change: thêm một boundary case và cập nhật đúng layers/tests.

Tuần 7 - Processing, lifecycle và issues: UC-15-19

Outcome

- Nắm chắc state machine và guards.
- Debug re-render/timing/race bằng trace và evidence.
- Hoàn thành processing queue, transitions và issue lifecycle theo canonical rules.

Teaching layer

- State-transition testing và invalid-transition matrix.
- Pessimistic order lock, serialized command và rollback mental model.
- Actionability/re-render nâng cao; trace, screenshot, video, console và request evidence.
- Network observation/interception ở mức thực dụng.
- Retry signal và flake taxonomy.

Canonical Laundry transfer

- UC-15-19.
- BR-15-16, BR-19-23.
- Chỉ forward transitions; cancelled/completed terminal.
- Open issue chặn ready/completed; resolution không rollback order status.
- Queue sort theo promise/urgency, không theo primary key.

Company transfer

Điều tra ba failure thật hoặc seeded: locator drift, async state và data/environment dependency. Luôn ghi hypothesis/evidence trước repair.

Deliverable

- Transition + issue decision table.
- 10-15 service/browser tests gồm invalid transitions.
- Ba failure investigation notes.
- Một trace bundle có thể teach-back.

Gate

- Seeded locator change sửa bằng semantic contract, không bằng CSS mong manh.
- Seeded race: tìm đúng evidence trước patch.
- Giải thích lock/guard/rollback bằng một concurrent-command scenario.

Tuần 8 - SQL, immutable ledger và pickup: UC-20-24

Outcome

- Dùng SQL làm business oracle.
- Hiểu collection/refund/pickup dưới góc transaction và concurrency.
- Làm đủ payment history/receipt/refund/pickup flows.

Teaching layer

- `SELECT`, filter, join, group, aggregate, `NULL`, stable ordering.
- Transaction, commit/rollback và read-after-lock.
- Database oracle vs coupling vào implementation.
- Read-only safety trên môi trường công ty.
- Money boundary, immutable ledger và derived values.

Canonical Laundry transfer

- UC-20-24.
- BR-24-31.
- Tự tính collection, refund, net paid, remaining, refund due và customer due.
- Không persist duplicate paid/refunded/remaining totals.
- Completed order không nhận transaction mới; pickup cần ready + zero remaining + no open issue.

Company transfer

Thiết kế một test triangle:

`UI action` → `HTTP/API evidence` → `database state`

Mặc định query read-only nếu công ty chưa cấp quyền write/test DB.

Deliverable

- Query workbook 10-12 câu.
- Money oracle table với boundary/negative/concurrency cases.
- UI/service/repository proof cho UC-20, UC-23 và UC-24.
- Một concurrent full-balance collection test hoặc precise design nếu environment chưa hỗ trợ.

Gate

- Tự viết join và aggregate query.
- Giải thích tại sao ledger immutable và derived balance ngăn drift.
- No-agent: tính đúng expected state cho một sequence collection/refund/pickup mới.

Tuần 9 - Public tracking, dashboard, reports, settings và pickup schedule: UC-25-30

Outcome

- Hoàn tất catalogue UC-01-30 về mặt hiểu biết và vertical coverage.
- Viết/đọc report queries với shop-time boundary đúng.
- Test data privacy, stable ordering, empty states và aggregates.

Teaching layer

- Half-open date range và timezone-safe testing.
- Aggregate/report oracle, grouping key và pagination/order stability.
- Data privacy và same-mismatch response.
- Clock injection/fixed-time testing.
- Browser/API/SQL evidence cho reporting features.

Canonical Laundry transfer

- UC-25-30.
- BR-32-36, đồng thời áp BR-07, BR-09, BR-19 và BR-30.
- Public tracking cần exact order code + normalized snapshot phone, không leak raw IDs/internal notes.
- Revenue dựa trên payment transaction time, không dựa order creation/status.
- Performance cohort và pickup schedule tuân exact query contracts.

Company transfer

Chọn một search/report/list feature. Thiết kế filters, empty state, date boundary, stable sorting và access-control tests.

Deliverable

- UC-25-30 traceability bundle.
- Report/query oracle workbook.
- Tests tại timezone/date boundary bằng fixed clock/data.
- Public tracking privacy tests.

Gate

- Tự giải thích half-open range và shop-local grouping.
- Phát hiện một report query dùng sai date/status basis.
- Random date change: dự đoán đúng affected query/test/UI evidence.

Tuần 10 - Cross-cutting CI/CD, persistence và canonical integrity

Outcome

- Chạy full proof ngoài laptop.
- Audit BR-37-39 và mọi cross-cutting invariant.
- Dùng Docker vừa đủ để môi trường không cản trở học/test.
- Phân loại pipeline failure bằng artifact.

Teaching layer

- CI trigger/job/step/runner, cache, browser dependencies, env/secrets và exit code.
- Report/trace artifact; deliberate red build và triage.
- Docker image/container/port/env/volume/log/compose ở mức vận hành.
- Flyway-owned schema, Hibernate validation, historical retention và traceability.
- Test portfolio theo layers: service, web, repository/PostgreSQL và end-to-end.

Canonical Laundry transfer

- BR-37-39.
- Audit đủ 30 UCs, 39 BRs, routes/forms, Controller/service, nine tables, migrations và minimum tests.
- Xác nhận không có JSP/Servlet/JDBC adapter, React/JWT/REST-first UI hoặc scope creep.
- Nếu project dùng Compose, chạy Spring Boot/PostgreSQL stack và đọc log; không đổi architecture chỉ để có Docker.

Company transfer

Đọc pipeline team nếu được phép. Xác định local-vs-CI differences, artifact location và triage ownership.

Deliverable

- Một CI workflow xanh từ clean checkout.
- HTML report + trace artifact.
- Một deliberate red-build triage note, sau đó restore green.
- Canonical coverage matrix và audit findings.
- Docker quick-run note nếu environment dùng container.

Gate

- Phân biệt test failure, product failure, dependency failure và environment/config failure.
- Từ một BR bất kỳ, chỉ ra affected screen/route/service/table/test/doc.
- Run/stop/log compose stack cơ bản nếu applicable.

Tuần 11 - Full canonical delivery, defense và automation capstone

Outcome

- Hiểu toàn bộ canonical Laundry, không chỉ phần được giao.
- Có automation capstone chứng minh test design, Playwright, API/HTTP, SQL, Git và CI.
- Chịu được random change và defense cross-question.

Teaching layer

- Risk-based regression selection.
- Traceability audit và Definition of Done cho từng UC.
- Defense storytelling: business rule → code → persistence → automated proof.
- Maintainability, fixture/data strategy và known limitations.
- Confidentiality-safe evidence; không polish CV/application trong giờ core.

Full-canonical sweep

Chọn ngẫu nhiên ít nhất một UC từ mỗi nhóm:

- access/people;
- catalogue/customer;
- order core;
- lifecycle/issues;
- money/pickup;
- tracking/reports/settings.

Với mỗi UC, trace full chain và trả lời affected BRs, invalid cases, transaction/time boundary và minimum tests.

Deliverable

- Automation capstone 25-35 meaningful tests hoặc equivalent evidence từ company/Laundry + sanitized public demo.
- Full canonical defense deck/notes.
- Gap list: missing code, test, doc, diagram, data hoặc understanding.
- Hai mock defenses và hai seeded-failure drills.
- Optional CV evidence log update tối đa 30 phút; không có application tracker trong curriculum.

Gate

- Người khác clean-clone và chạy smoke/CI evidence được.
- Nói một UC ngẫu nhiên 8-10 phút không nhìn tài liệu.
- No-agent random change: xác định impact trước, sửa/test sau.
- Không có canonical rule nào chỉ “biết tên” mà không nêu observable outcome/test oracle.

Tuần 12 - Consolidation: independent transfer + full SWP defense

Outcome

- Chứng minh transfer trên một feature lạ.
- Debug dưới áp lực vừa phải mà không phụ thuộc AI.
- Defense toàn bộ Laundry và demo automation evidence mạch lạc.
- Chọn được nhánh kỹ năng 30 ngày tiếp theo mà không chạy theo mọi tool.

Graduation simulation

Part A - Test design, 30 phút

Nhận một feature/requirement chưa học trước. Tạo risk list, questions, test conditions và automation candidates.

Part B - Random change, 45 phút

Nhận một thay đổi UI/behavior nhỏ. Cập nhật test và giữ suite ổn định.

Part C - Seeded failure, 30 phút

Debug một failure có trace/log. Viết hypothesis và evidence trước patch.

Part D - Technical explanation, 20 phút

Giải thích một test Playwright, một API check, một SQL oracle và một CI failure.

Part E - SWP defense, 20 phút

Trace một UC ngẫu nhiên từ screen đến database/test và nêu affected BRs.

Part F - Skill handoff, 20 phút

Demo automation evidence, kể một debugging/transfer story và chọn một nhánh kỹ năng tiếp theo dựa trên task/JD mà người học tự theo dõi.

Deliverable

- Clean-clone run xanh.
- Graduation evidence bundle.
- Final SWP rehearsal notes.
- Automation capstone v2.
- 30-day skill-improvement plan; việc tìm và nộp job không nằm trong curriculum.

Gate

Pass khi người học có thể tự thiết kế, tự viết biến thể, tự debug và tự giải thích. AI có thể review sau khi người học đã đưa hypothesis/evidence, nhưng không thay thế phần đầu của bài.

7. CI/CD hay Docker trước?

Verdict

Học CI/CD trước và sâu hơn Docker.

Lý do:

- Playwright chỉ trở thành regression signal có giá trị khi chạy lặp lại ngoài máy người viết.
- Job automation thường yêu cầu tích hợp test vào pipeline hoặc ít nhất biết đọc pipeline failure.
- CI buộc người học xử lý environment, dependency, headless behavior, exit code, report và artifact.
- Docker hữu ích để tái tạo môi trường, nhưng học networking/image optimization quá sớm không sửa được test design yếu, missing `await` hay locator mong manh.

Mức CI/CD phải đạt

- Hiểu trigger/job/step/runner.
- Chạy install + test từ clean environment.
- Dùng env/secrets đúng mức cơ bản.
- Upload report/trace artifact.
- Cố tình tạo red build, triage và restore green.
- Đọc log để phân loại lỗi test/product/environment.

Mức Docker phải đạt

- Hiểu image vs container.
- `run`, `stop`, `logs`, `port`, `environment` và `volume`.
- Dùng `docker compose` để khởi động một stack có sẵn.
- Biết chỗ dữ liệu được giữ và cách đọc health/log.

Chưa cần trong 12 tuần

- Multi-stage image optimization.
- Kubernetes.
- Production networking/security hardening.
- Cloud deployment architecture.
- Tự containerize mọi thứ chỉ để thêm logo Docker vào CV.

Khi nào đảo thứ tự?

Chỉ đưa Docker lên sớm hơn nếu:

- internship task bị block vì không chạy được môi trường container;
- SWP team đã thống nhất Docker Compose là cách local setup;
- JD/phòng vấn cụ thể trong bảy ngày yêu cầu Docker operational knowledge.

8. Portfolio architecture

Mục tiêu của portfolio

Trong v4, portfolio trước hết là **skill evidence repository**, không phải project marketing chiếm giờ học. Nếu người học muốn public sau này, chỉ cần làm sạch và bổ sung presentation layer. Nó phải trả lời năm câu hỏi của reviewer/mentor:

1. Ứng viên có biết chọn cái gì đáng test không?
2. Test có ổn định và dễ đọc không?
3. Ứng viên có debug được failure không?
4. Suite có chạy ngoài laptop không?
5. Ứng viên có giao tiếp rõ ràng và trung thực không?

SUT strategy

Ưu tiên theo thứ tự:

1. Một public/demo SUT ổn định, được phép automate.
2. Laundry nếu repo/app được phép dùng làm evidence và đã đủ ổn định; không mặc định phải public.
3. Hai SUT nhỏ: Laundry cho UI/Spring trace, demo API riêng cho API skill.

Không commit company code, credentials, domain nội bộ, screenshot chứa dữ liệu nhạy cảm hoặc test data nhận diện khách hàng.

Không thêm REST endpoints hoặc frontend thứ hai vào canonical Laundry để portfolio “trông hiện đại”.

Cấu trúc repo gợi ý

```

qa-automation-portfolio/
├── README.md
├── package.json
├── playwright.config.ts
├── .env.example
├── tests/
│   ├── ui/
│   │   ├── public/
│   │   ├── auth/
│   │   └── workflow/
│   ├── api/
│   └── fixtures/
├── support/
│   ├── data-builders/
│   └── domain-helpers/
├── sql/
│   └── validation_queries.sql
├── docs/
│   ├── test-strategy.md
│   ├── risk-matrix.md
│   ├── failure-investigations/
│   ├── bug-reports/
│   └── decisions/
├── .github/
│   └── workflows/
│       └── playwright.yml

```

Không cần tạo folder hoặc abstraction nếu chưa có nội dung thật. Cấu trúc phục vụ reader, không phục vụ việc khoe architecture.

Coverage tối thiểu có ý nghĩa

- 5-8 smoke tests cho critical path.
- 8-12 negative/boundary/role/state tests.
- 8-12 API tests trên target phù hợp.
- Một data-driven example có lý do.
- Một auth/data fixture.
- Một traceable failure investigation.
- Một cross-browser smoke job hoặc project.
- Một CI workflow có artifact.
- 10-12 SQL queries hoặc data-oracle notes.

Số lượng là hướng dẫn, không phải KPI. Hai mươi lăm test có reasoning tốt mạnh hơn một trăm test lặp cùng một happy path.

README bắt buộc trả lời

- SUT và business problem là gì?
- Risk nào được ưu tiên?
- Cái gì được và không được automate?
- Cách setup/run smoke/full/API tests.
- Data/auth strategy.
- CI/report/artifact ở đâu?
- Known limitations.
- Một hoặc hai debugging stories.
- Những gì là school project, personal work và internship learning.

Demo 3-5 phút

```
30 giây: problem + risk
60 giây: test strategy
60 giây: một stable Playwright test
45 giây: một failure/trace story
30 giây: CI result
30 giây: limitation + next improvement
```

Demo là gate diễn đạt kỹ thuật, không bắt buộc đăng mạng hoặc dùng nó để nộp hồ sơ trong 12 tuần.

9. Skill evidence log - CV use là tùy chọn

Curriculum không viết CV hay tìm/nộp job thay người học. Tuy vậy, không đợi tới lúc tự làm CV mới cố nhớ lại mình đã làm gì. Mỗi tuần dành tối đa 10-15 phút ghi evidence kỹ thuật ngay khi còn mới; log này chủ yếu phục vụ reflection, teach-back và gate, sau này có thể chuyển thành CV bullet.

Template

Evidence ID: EV-YYYY-Wxx-NN

- Context: Internship / SWP391 / Personal portfolio
- Feature or risk:
- My responsibility:
- Expected behavior:
- Actual problem or task:
- Hypotheses considered:
- Actions I personally performed:
- Tools used:
- Evidence produced: PR / test / trace / bug / query / report
- Result:
- Metric, if real and available:
- What I learned:
- What I would improve next:
- Confidentiality-safe phrasing for CV:
- Link to public evidence, if allowed:

Metric hợp lệ

- Số critical flows được cover.
- Số flaky tests được ổn định với phương pháp rõ ràng.
- Runtime trước/sau nếu đã đo.
- Reproduction rate của bug.
- Số manual steps giảm nếu có bằng chứng.
- Pipeline pass rate trong một cửa sổ được xác định.
- Defect được phát hiện trước release, nếu công ty cho phép nhắc ở mức tổng quát.

Không tự bịa “tăng chất lượng 70%” hoặc “giảm 90% effort” khi không có baseline và measurement.

Chuyển evidence thành CV bullet khi người học cần

Công thức:

Action + scope/risk + method/tool + observable result/evidence

Ví dụ an toàn:

Designed and automated Playwright checks for critical authenticated web flows using TypeScript, semantic locators, isolated test data, and trace-based failure analysis.

Ví dụ có metric, chỉ dùng khi thật:

Stabilized 6 flaky end-to-end checks by replacing fixed waits with observable UI/network conditions, achieving 20 consecutive local runs without an unexpected failure.

Ví dụ SWP:

Traced and tested server-rendered Spring MVC order/payment flows from form binding through transactional services, PostgreSQL persistence, and business-rule assertions.

Phải ghi rõ context trong CV nếu đó là academic hoặc personal project. Phần chuyển bullet là optional và không được lấy thời gian khỏi JS/TS, Playwright hoặc SWP gates.

10. Interview drills

Nhịp cố định

- Mỗi tuần: một teach-back 10 phút.
- Từ tuần 4: một seeded failure 20-30 phút.
- Từ tuần 8: một mock segment 30 phút.
- Tuần 11-12: mock interview đầy đủ 60 phút.

QA/test-design drill

Cho một feature như login, search, payment hoặc upload. Trong 15 phút:

1. hỏi clarification;
2. nêu risks;
3. chia test conditions;
4. chọn boundary/negative cases;
5. quyết định cái gì automate và cái gì chưa;
6. nêu oracle/evidence.

JavaScript/TypeScript drill

- Dự đoán output của function có array/object/reference.
- Sửa lỗi `return`, scope hoặc mutation.
- Tìm missing `await`.
- Narrow một union/`unknown` an toàn.
- Đọc một TypeScript compiler error và diễn giải bằng lời thường.

Playwright drill

- Chọn locator và bảo vệ lựa chọn.
- Sửa test dùng `sleep`.
- Giải thích auto-wait vs assertion wait.
- Thiết kế isolated data.
- Debug trace và phân loại product/test/environment failure.
- Nêu lúc nào dùng API setup thay UI setup.

API/HTTP drill

- Thiết kế tests cho create/update/read resource.
- Phân biệt 400, 401, 403, 404, 409, 422 theo contract cụ thể.
- Kiểm tra không chỉ status mà body, side effect, authorization và data consistency.
- Giải thích cookie/session/CSRF so với token auth ở mức junior.

SQL drill

- Viết join lấy order và transactions.
- Aggregate collection/refund.
- Tìm duplicate hoặc orphan-like anomaly.
- Giải thích half-open date range.

- Dự đoán effect của `NULL` và duplicate rows.

Git/CI drill

- Đọc diff và tìm accidental secret/change.
- Giải thích branch/commit/PR flow.
- Pipeline đỏ nhưng local xanh: lập hypothesis list.
- Tìm report/trace artifact và chọn evidence tiếp theo.

SWP/Spring drill

- Trace một UC ngẫu nhiên từ route đến table/test.
- Phân biệt Controller validation và service business guard.
- Giải thích `@Transactional`, rollback, pessimistic lock và PRG.
- Chỉ ra tác động khi đổi một BR lên route/form/test/document.

Behavioral drill

Chuẩn bị STAR stories cho:

- một bug khó tái hiện;
- một lần giả thuyết đầu tiên sai;
- một test flaky;
- một requirement mơ hồ;
- một deadline/team conflict;
- một skill tự học và chuyển được sang task thật.

No-agent drill

Trong live drill:

1. không nhờ AI viết patch đầu tiên;
2. ghi expected/actual/hypothesis/evidence;
3. thực hiện một experiment nhỏ;
4. sau đó mới dùng AI để review reasoning hoặc patch.

11. English lane

Baseline tự báo cáo khoảng IELTS 6.5 dù chưa có chứng chỉ, nên v4 không mở lại grammar/general-English syllabus. English được dùng như phương tiện đọc docs, viết evidence và giải thích kỹ thuật.

Nhịp tối thiểu

Hai hoặc ba phiên 15-20 phút mỗi tuần:

1. **Read:** đọc một đoạn official docs, viết 3-5 câu tóm tắt.
2. **Write:** viết bug report, PR summary hoặc test rationale bằng English.
3. **Speak:** nói 2-3 phút về một test/failure, ghi âm và tự nghe lại.

Vocabulary theo phase

Tuần 1-3

Expected, actual, reproduce, precondition, boundary, assertion, locator, timeout, flaky, deterministic.

Tuần 4-6

Fixture, isolated, authenticated, authorized, request, response, payload, schema, query, transaction, rollback.

Tuần 7-9

Trace, artifact, pipeline, regression, root cause, mitigation, concurrency, lock, persistence, redirect.

Tuần 10-12

Risk, trade-off, coverage, limitation, impact, ownership, collaboration, maintainability, follow-up.

Weekly spoken prompts

- What risk does this test cover?
- Why did you choose this locator?
- How did you determine whether the failure was in the product or the test?
- What evidence changed your hypothesis?
- What would you automate next, and why?
- What is one limitation of your current solution?

Gate

Không cần accent hoàn hảo hoặc luyện thi chứng chỉ trong 12 tuần. Cần nói chậm, có cấu trúc, dùng đúng evidence và hiểu câu hỏi. Cuối tuần 12 phải demo automation evidence 3-5 phút bằng English hoặc song ngữ.

12. Branching rules theo deadline và thực tế internship

Rule A - SWP red zone

Kích hoạt khi milestone/defense còn dưới 14 ngày hoặc team đang bị block bởi phần người học sở hữu.

Phân bổ tạm thời:

- 50% SWP vertical slice và defense evidence.
- 35% internship task trực tiếp.
- 15% career maintenance.

Dừng học tool mới. Dừng Laundry làm SUT cho Playwright/SQL/trace nếu có thể. Không mở nhánh performance, Docker sâu hoặc portfolio polish.

Rule B - Internship delivery red zone

Kích hoạt khi task thật có deadline dưới ba ngày.

- 70% task và prerequisite trực tiếp.
- 20% learning repair đúng chỗ đang block.
- 10% SWP maintenance.

Sau delivery, ghi evidence log và quay lại gate đang dở. Không reset curriculum.

Rule C - Interview red zone

Kích hoạt khi có interview trong bảy ngày.

- Parse JD thành must-have/nice-to-have.
- Chọn ba gap lớn nhất có thể cải thiện bằng evidence.
- Thay tối đa hai lesson kế tiếp bằng targeted drills.
- Không thêm feature portfolio chỉ để tăng số lượng.
- Docker chỉ được ưu tiên nếu JD hỏi trực tiếp.

Rule D - Foundation gate failed

Nếu cuối tuần 2 chưa tự viết được function xử lý array/object hoặc chưa đọc được TypeScript error:

- chạy repair cycle ba ngày;
- mỗi ngày 30-45 phút JS/TS + một test Playwright nhỏ;
- hoãn fixture/framework abstraction;
- không hoãn toàn bộ contact với browser automation.

Rule E - Company stack khác Playwright + TS

Nếu công ty giao Python/Selenium, Java/Selenium hoặc framework khác:

- ưu tiên stack công ty cho task thật;
- giữ tối thiểu hai phiên Playwright + TS mỗi tuần cho portfolio;
- không cố học sâu hai framework trong cùng một phase;
- sau tuần 6 đánh giá lại job target dựa trên evidence thật.

Rule F - Laundry chưa runnable

Không chờ app hoàn thiện mới học testing:

- hoàn thiện requirement/risk/test model;
- trace từ canonical docs;
- automate kỹ thuật trên demo SUT;
- chuyển tests sang Laundry theo từng vertical slice khi route xuất hiện.

Rule G - Mất nhịp

Nghỉ 1-2 ngày

Đọc last evidence log, chạy một smoke test, tiếp tục task nhỏ nhất.

Nghỉ 3-6 ngày

Làm retrieval quiz 20 phút, chọn lại WIP = 1 và hoàn thành một required deliverable trước stretch.

Nghỉ từ 7 ngày

Không học lại từ tuần 1. Chạy calibration lite:

- một JS/TS exercise;
- một Playwright reading/debug task;
- một SWP reverse explanation.

Repair đúng gap, rồi quay về gate gần nhất chưa qua.

13. Sau 12 tuần nên học sâu automation hay rẽ sang CI/CD/Docker?

Việc tìm JD và nộp hồ sơ do người học tự quản lý, không phải deliverable của curriculum. Market evidence chỉ giúp quyết định nhánh kỹ năng tiếp theo.

Default recommendation

Nếu chưa có yêu cầu khác từ internship/job target, dành 30 ngày tiếp theo để **học sâu automation quality**:

- framework maintainability dựa trên duplication/risk thật;
- API setup và contract checks;
- database/test-data strategy;
- flake measurement và failure triage;
- CI parallelism/sharding/reporting ở mức suite thật;
- thêm một unfamiliar-SUT capstone.

Không nhảy sang Docker/Kubernetes như một nghề riêng chỉ vì thấy chúng xuất hiện trong roadmap DevOps.

Nhánh CI/CD sâu hơn khi

- đã có suite ổn định và đang bị chậm/khó vận hành trên pipeline;
- internship giao ownership pipeline;
- JD mục tiêu hỏi Jenkins/GitLab/GitHub Actions rõ ràng;
- người học thích release engineering và điều tra environment failures.

Nội dung tiếp theo có thể là matrix jobs, shard, cache, service container, test selection, flaky-test quarantine policy và artifact/report integration.

Nhánh Docker sâu hơn khi

- local/CI environment đang là bottleneck thật;
- cần dựng app + DB + dependencies lặp lại;
- team giao Dockerfile/Compose ownership;
- JD cụ thể đòi operational container knowledge.

Nếu không có trigger trên, Docker vẫn ở mức run/log/compose và thời gian được đầu tư vào Playwright/API/SQL/debug.

Nhánh khác chỉ khi có evidence-driven trigger

- Selenium/Java hoặc Python: công ty/JD mục tiêu dùng thật.
- Mobile/Appium: có product/mobile role cụ thể.
- Performance: đã vững functional test và có latency/throughput problem cụ thể.
- Security testing: có scope/mentor phù hợp, không tự pentest hệ thống không được phép.

14. Calibration questionnaire trước khi khóa v4

Dữ kiện đã khóa

- Internship còn gần ba tháng.
- SWP391 bắt đầu sau khoảng ba tuần và kéo dài hai tháng.
- Mục tiêu không chỉ pass mà là tự làm và hiểu toàn bộ canonical Laundry.
- Baseline được xem là true zero vì bốn năm phụ thuộc AI.
- Khu vực việc làm tự theo dõi: Cần Thơ và TP.HCM.
- English tự đánh giá khoảng IELTS 6.5, chưa có chứng chỉ.
- Người học tự tìm/nộp job; v4 chỉ huấn luyện kỹ năng và readiness.

Các câu còn lại dùng để đổi tỷ trọng, bài tập và deadline. Không cần đợi trả lời hết mới bắt đầu Week 1, nhưng nên hoàn thành trong ba ngày đầu.

A. Thời gian và năng lượng

1. Internship hiện bao nhiêu giờ mỗi tuần? Có phải full-time không?
2. Thực tế có thể dành bao nhiêu giờ tự học ngoài internship: 4, 8 hay 12 giờ?
3. Khung giờ nào thường tập trung tốt nhất?
4. Ngoài AI dependence, nguyên nhân chính làm mất nhịp là gì: mệt, không biết học gì, task quá khó, thiếu môi trường hay deadline chồng nhau?
5. Có ngày nào bắt buộc không học được không?

B. SWP391

6. Ngày bắt đầu chính xác, proposal, implementation review, document submission và defense là ngày nào?
7. Team hình thành chưa; cách chia vertical slice/ownership dự kiến ra sao?
8. Codebase/template/repo Spring Boot đã tồn tại hay sẽ scaffold từ đầu?
9. Phần nào người học sợ nhất: Java/Spring, database, test, document, diagram, Git teamwork hay defense?
10. Có được phép dùng Laundry làm public evidence không, hay repo phải private?

C. Internship

11. Công ty dùng Playwright với TypeScript, JavaScript hay ngôn ngữ khác?
12. Hiện người học đang làm manual test, viết automation, maintain suite hay chỉ học onboarding?
13. Có quyền đọc/chạy CI, API docs, test database và source code không?
14. Tool thực tế là GitHub/GitLab/Bitbucket, Jira/Azure DevOps, Postman hay tool nào khác?
15. Task nào đã hoàn thành mà có thể dùng làm sanitized learning evidence?
16. Công ty có mentor/reviewer không? Feedback hiện tại về điểm yếu là gì?

D. Baseline kỹ thuật

17. Không nhờ AI, có thể tự viết một function JavaScript có parameter/return không?
18. Không nhờ AI, có xử lý được array/object và `map/filter` không?
19. Có thể tự giải thích `async/await` và missing `await` không?
20. Có tự viết một Playwright test từ file trống không nhìn tutorial không?
21. Có đọc được HTTP request/response trong DevTools không?
22. Có dùng Git branch/commit/PR chưa?

23. Có viết được `SELECT ... WHERE ...` và một `JOIN` chưa?
24. Có từng tự debug pipeline failure chưa?

E. Job target

25. Chấp nhận role Manual/Hybrid QA có automation growth path không, hay chỉ muốn pure automation?
26. Mức lương tối thiểu và thời điểm cần có thu nhập là gì?
27. Có thể public GitHub/evidence repo và quay demo video không?

F. English

28. Có thể đọc official docs mà không dịch từng câu không?
29. Có viết được bug report ngắn bằng English không?
30. Có thể giải thích một test/failure trong hai phút không?
31. Technical speaking là điểm mạnh hay điểm yếu so với reading/listening?

G. Môi trường

32. Máy có chạy ổn Playwright browsers, Java 21, PostgreSQL và Docker không?
33. Có tài khoản GitHub/GitLab cá nhân chưa?
34. Có giới hạn mạng/VPN/quyền cài tool ở máy công ty không?
35. Có dữ liệu hoặc source nào tuyệt đối không được đưa ra ngoài không?

Cách dùng câu trả lời

- Baseline hiện đã là true zero, nên Foundation lane tuần 1-3 mặc định chiếm ít nhất 50%; questionnaire quyết định có cần repair cycle bổ sung hay không.
- Nếu internship đã giao Playwright thật: company transfer trở thành deliverable chính, portfolio dùng bản sanitized hoặc demo tương đương.
- Vì SWP bắt đầu ở tuần 4 và kéo dài tám tuần, full canonical integration là default, không phải nhánh optional.
- Nếu cần thu nhập rất gấp, người học tự mở rộng target Manual/Hybrid QA; curriculum vẫn giữ skill gates.
- Với English khoảng IELTS 6.5, ưu tiên technical speaking và bug/PR writing, không luyện thi chứng chỉ.
- Nếu máy yếu: dùng một browser project local, CI cho cross-browser; Docker chỉ bật khi thật sự cần.

15. Definition of ready sau 12 tuần

Ready cho internship Playwright task

- Đọc được existing test và hiểu data/state.
- Chuyển acceptance criteria thành test conditions.
- Viết locator semantic và web-first assertion.
- Không dùng sleep để che timing.
- Tự điều tra trace/log trước khi xin patch.
- Làm change qua Git/PR.
- Nói rõ confidence và limitation.

Ready cho SWP391 defense

- Giải thích canonical scope và các exclusion quan trọng.
- Trace UC từ actor/screen tới route, security, Controller, service, repository/table và tests.
- Giải thích lifecycle, money ledger, issue guard, transaction/lock và shop time.
- Chỉ ra code/document/test bị ảnh hưởng khi business rule đổi.
- Không coi “AI code chạy xanh” là bằng chứng đã hiểu.

Ready về kỹ năng cho Junior QA/QC

- Có skill evidence đủ để tự chuyển thành CV khi cần và không khai khống.
- Có evidence repo hoặc demo thay thế hợp lệ; không bắt buộc public trong curriculum.
- Có test-design, Playwright, API, SQL, Git và CI examples.
- Có ít nhất năm STAR stories.
- Qua được random-change và seeded-failure drill.
- Có thể trình bày bản thân/test/failure bằng English cơ bản.

16. Quy tắc giữ v4 trung thực và hiệu quả

1. Không biến roadmap thành checklist logo công nghệ.
2. Không thêm Docker/Kubernetes trước khi test fundamentals và Playwright core qua gate.
3. Không thêm REST/React/JWT hoặc feature ngoài canonical Laundry để phục vụ portfolio.
4. Không dùng số test làm proxy duy nhất cho chất lượng.
5. Không đưa code/data mật của internship ra ngoài.
6. Không để AI viết lời giải đầu tiên trong prediction, random-change và debug gate.
7. Không restart sau khi nghỉ; repair đúng gap.
8. Không đợi “đủ giỏi” mới ghi skill evidence; việc nộp hồ sơ do người học tự quản lý ngoài curriculum.
9. Không chỉ luyện happy path; role, boundary, invalid state và failure evidence là bắt buộc.
10. Không tuyên bố job-ready nếu chưa thể giải thích, sửa biến thể và debug độc lập.

17. One-page priority card

Khi không biết hôm nay nên học gì, dùng thứ tự này:

1. Có deadline thật trong 72 giờ không?
Có → làm task trực tiếp + prerequisite nhỏ nhất.
Không → tiếp tục.
2. Gate hiện tại đã qua chưa?
Chưa → làm required lab/failure drill của gate.
Rồi → tiếp tục.
3. SWP milestone còn dưới 14 ngày không?
Có → làm vertical slice/defense evidence.
Không → tiếp tục.
4. Automation capstone đã có clean-clone CI evidence chưa?
Chưa → làm artifact gần nhất.
Rồi → tiếp tục.
5. Có interview trong bảy ngày không?
Có → targeted technical drill; người học tự quản lý application.
Không → làm delayed retrieval hoặc English lane.

Với mục tiêu có việc sớm, default luôn là:

Học sâu hơn về testing + Playwright + debug, rồi bổ sung API/SQL/Git/CI. Docker chỉ vừa đủ để không bị môi trường chặn; không dùng nó để trì hoãn việc tạo evidence tuyển dụng thật.